DGD721 · Practice Sheet · Week 5

Variables · conditions · switch · loops · methods

How to work

• Work on your own. Create a new console app called **Week05**.
• Write **all tasks in one Program.cs**. Before each task, write a comment like `// ===== B1 =====`.
• Put each task inside its own { } block, so you can use the same variable names (hp, gold…) again.
• Part A: write your answers as comments at the top, **before** running any code.
• A task that doesn't work? Comment it out and write `// not finished`. Your file must still run.
• ★ = warm-up · ★★ = normal · ★★★ = challenge. Do them in order.

Submit on GitHub

In your **DGD721-name-surname** repository, create a folder **week05** and upload your **Program.cs**. Upload it by the end of class.

Part A · Predict the output (no running the code!)

A1 What does this print? ★

```
int x = 11;
int y = 4;
Console.WriteLine(x / y + x % y);
```

My answer: _____

A2 What does this print? ★

```
int hp = 45;
string msg = hp > 50 ? "Healthy" : hp > 20 ? "Hurt" : "Critical";
Console.WriteLine(msg);
```

My answer: _____

A3 What does this print? ★

```
int total = 0;
for (int i = 1; i <= 4; i++)
{
    if (i % 2 == 0) total += i;
    else total -= 1;
}
Console.WriteLine(total);
```

My answer: _____

A4 What does this print? ★

```
int Twice(int n)
{
    return n * 2;
}

int a = Twice(3);
a = Twice(a + 1);
Console.WriteLine(a);
```

My answer: _____

Part B · Write the code

B1 Damage calculator ★

• Create three variables: **baseDamage = 40**, **armor = 15**, **level = 4**.

• Damage is **baseDamage − armor**. If the level is **5 or more**, add a **10** bonus.

• Damage can never be less than **1**.

• Print the damage with a label: **30 or more** → “Strong hit”, **10–29** → “Normal hit”, **below 10** → “Weak hit”.

• Test it again with level = 6, and with armor = 50.

Expected output

```
Damage: 25 (Normal hit)
```

B2 The weapon shop ★

• Create **gold = 120** and **choice = 2**.

• Use a **switch** on choice: 1 = Sword (100 gold), 2 = Bow (80 gold), 3 = Potion (25 gold). Any other number: print “Invalid choice”.

• If the player has enough gold, buy it and print the gold left. If not, print how much more gold is needed.

• Tip: inside the switch, only save the item name and the price. Decide about buying after the switch.

Expected output

```
You bought a Bow! Gold left: 40
```

B3 Monster waves ★★

• Use a **for** loop for waves **1 to 30**.

• If the wave number can be divided by **3**: “Slime”. By **5**: “Bat”. By **both 3 and 5**: “BOSS”. Otherwise: “empty”.

• Tip: use **%**. Think carefully about the order of your conditions!

Expected output

```
Wave 1: empty
Wave 2: empty
Wave 3: Slime
Wave 4: empty
Wave 5: Bat
Wave 6: Slime
...
Wave 15: BOSS
```

B4 XP grind ★★

• Start with **level = 1**, **xp = 0**, **fights = 0**.

• Use a **while** loop: keep fighting until the hero reaches **level 5**. Every fight gives **35 xp**.

• When xp reaches **100 or more**: subtract 100 from xp, add 1 to level, and print “LEVEL UP! Now level X”.

• After the loop, print how many fights it took and how much xp is left.

Expected output

```
LEVEL UP! Now level 2
LEVEL UP! Now level 3
LEVEL UP! Now level 4
LEVEL UP! Now level 5
Reached level 5 after 12 fights (xp left: 20)
```

B5 Keep it in range: Clamp() ★★

• Write a method: **int Clamp(int value, int min, int max)**.

• If value is smaller than min, return min. If it is bigger than max, return max. Otherwise return value.

• Test it: Clamp(150, 0, 100), Clamp(−20, 0, 100), Clamp(55, 0, 100).

• Then use it: **hp = 80**, the hero drinks a potion (+50). Use Clamp so hp stays between 0 and 100.

Expected output

```
100
0
55
HP after potion: 100
```

B6 Medal ranks: GetRank() ★★

• Write a method: **string GetRank(int score)**. 1000 or more → “Gold”, 500 or more → “Silver”, 1 or more → “Bronze”, otherwise → “No medal”.

• Use a **for** loop: score goes from **0 to 1250**, increasing by **250** each time. Print each score with its rank.

Expected output

```
Score 0: No medal
Score 250: Bronze
Score 500: Silver
Score 750: Silver
Score 1000: Gold
Score 1250: Gold
```

B7 Boss battle ★★★

• Write two methods: **int Attack(int hp, int damage)** (hp goes down, but never below 0) and **bool IsAlive(int hp)**.

• Player: **150 HP**, hits for **25**. Boss: **120 HP**, hits for **25**, but on every **3rd turn** it hits for **double** damage.

• Every turn the player attacks first. If the boss falls, the battle ends immediately (the boss cannot hit back).

• Use a loop while both are alive. Print every turn. At the end, print the winner, the number of turns and the HP left.

Expected output

```
Turn 1 | Boss: 95 | Player: 125
Turn 2 | Boss: 70 | Player: 100
Turn 3 | Boss: 45 | Player: 50
Turn 4 | Boss: 20 | Player: 25
Turn 5 | Boss: 0
You win in 5 turns! HP left: 25
```

B8 Guess the number ★★★

• The secret number is **42**. The player has **5 tries**.

• Write a method: **string Hint(int guess, int secret)** that returns “Too low”, “Too high” or “Correct!”.

• Use a loop. Each try: ask for a number (Console.ReadLine + int.Parse), print the hint and the tries left.

• Stop when the guess is correct or when there are no tries left. Print a win or lose message.

• Bonus: make the secret random: **new Random().Next(1, 101)**.

Expected output

```
Guess (5 tries left): 50
Too high
Guess (4 tries left): 30
Too low
Guess (3 tries left): 42
Correct!
You found it in 3 tries!
```